



Technical Report

Skills Path Architecture for Agile Development *Set the definition of done for a skill in development*

SUBMITTED BY
Maamoun Youssef

Jira Clone
2026-06-01

Abstract

This report describes how the development skills directory in this Salesforce/LWC project is shaped to automate the Development phase of the Agile SDLC for the developer — to set a repeatable definition of done for task development — while adapting to the project's architecture and rules. It is organized around the three challenges the architecture solves — reliability, consistency, and maintainability. The current design pushes every reusable rule down to a leaf level of guide files under task-skill/guide/. Each task-skill opens with a Mandatory-guides step that names exactly which guides it depends on, tagged required or optional, so the dependency graph is a shallow tree (orchestrator -> task-skill -> guide) instead of a mesh. Cross-cutting performance and governor-limit guards are not owned by any single task-skill; they are project-wide and triggered from CLAUDE.md.

1. The goal

This skill set is not a grab-bag of recipes. It targets one intersection of the SDLC and one role, under one set of constraints.

Question	Answer
Which SDLC phase does this automate?	The Development phase of Agile.
Who uses it?	The developer.
For what?	To implement a task (the bug flow is reserved for later).
Constrained by what?	The project's architecture and rules — LWC layering, Apex bulkification, soft-delete filter, required fields, and the Controller -> APIResponse -> Service -> DomainCorrectness layering.

Restated:

Build a skill that automates the Development phase of the Agile SDLC for the developer, that adapts to this project's architecture, rules, and framework, and that delivers reliability, consistency, and maintainability without architectural overhead.

The rest of this document is the architecture that meets that goal, organized by the three challenges it has to solve.

2. The three challenges (with examples)

2.1 Reliability — the right skill must fire even when the user didn't name it

Problem. A skill (or guide) only fires if the model picks it. If the user's prompt doesn't name a downstream guide, it gets silently skipped.

Solution. Make the call explicit inside the calling task-skill. Each task-skill's Step 0 — Mandatory guides names the leaf guides it depends on, so they fire regardless of wording.

Example. The user says "add this functionality to my parent" — nothing about layering, error handling, or state rules. The task-skill's Step 0 names those guides as mandatory, so they run silently before any interview question:

add-interactive-with-data-persistence-functionality-in-pather.md

Step 0 — Mandatory guides (required, no question):

lwc-path-architecture-guide · lwc-error-handling-guide ·
pather_lwc_state_management_checklist-guide

2.2 Consistency — same intent, same path, every time

Problem. With N skills directly addressable, the user's wording decides which one fires. Same intent, different phrasings, different paths.

Solution. One entry point — the orchestrator (triggered by "development run") — and an AskUserQuestion flow that turns the prompt into a fixed set of answers, so the user's intent is captured with 100% certainty.

Example. Any dev request enters agile-development-orchestrator-skill.md and answers the classification questions: bug vs. task, then A/B/C, then parent vs. child. Three different phrasings of "build a feature on a parent LWC" all land on the same row:

B — create new functionality / parent ->

add-interactive-with-data-persistence-functionality-in-pather.md

2.3 Maintainability — a new task can't fan out edits across every other task

Problem. When several task-skills need the same sub-step, in-lining it in each one means every change has to touch every copy.

Solution. Extract the shared sub-step into one leaf-level guide file under task-skill/guide/ and have each task-skill reference it from its Mandatory-guides step. The rule lives in exactly one file.

Example. Both add-interactive-with-data-persistence-functionality-in-pather.md and lwc-parent-event-handler-generator.md need to resolve "which Apex method backs this?".

Instead of asking that interview manually in each file, both depend on guide/apex-method-resolution-guide.md — the three-branch interview (existing-known / existing-find / create-new) lives in exactly one leaf guide.

3. Adaptation to project architecture and rules

The orchestrator does not invent rules — it routes work into task-skills, and each task-skill reaches the project's rules through the leaf guides it depends on. Two mechanisms keep this adaptation predictable.

3.1 The Mandatory-guides step: required vs optional

Every task-skill opens with a Step 0 — Mandatory guides that lists the leaf guides it depends on. Each guide is tagged, and the tag decides how it is read:

required — no question is asked. Look at the scope of the change: read and apply the guide when its trigger matches that scope (most run on every iteration); otherwise skip it and state the skip reason in the iteration message.

optional — a single AskUserQuestion decides whether the guide is read. Apply it only when the user answers yes; never apply it silently.

3.2 Project-wide cross-cutting skills live in CLAUDE.md, not in a task-skill

The performance/ and guard/ skills encode disciplines that apply to any Apex on the project, regardless of which task-skill produced it. They are NOT wired into a task-skill's Mandatory-guides step; CLAUDE.md decides when they fire. That keeps each cross-cutting rule in exactly one place and out of the per-task dependency graph — so adding a task-skill never multiplies edges to the guards. The bulkification chain (governor-limit guard -> bulk-SOQL rewrite -> method monitor) and the soft-delete filter are all triggered this way.

4. Current file layout (for reference)

force-app/skills/development/

```
├─ agile-development-orchestrator-skill.md    <- single entry ('development run')
├─ guard/                                     <- project-wide, triggered from CLAUDE.md
│   ├── apex-governor-limit-guard.md
│   └─ interview-discipline.md
├─ performance/                             <- project-wide, triggered from CLAUDE.md
│   ├── apex-bulk-soql.md
│   └─ apex-method-monitor.md
└─ task-skill/
```

- └─ create-new-parent-lwc-component.md <- pather router (no direct guides)
- └─ add-interactive-with-data-persistence-functionality-in-pather.md
- └─ lwc-parent-event-handler-generator.md
- └─ create-new-child-lwc-component.md
- └─ add-functionality-in-child-lwc-component.md
- └─ shared/
 - └─ ask-user-for-lwc-validation.md
- └─ guide/ <- leaf level (depends on, never depends)
 - └─ lwc-path-architecture-guide.md
 - └─ apex-path-architecture-guide.md
 - └─ apex-method-resolution-guide.md
 - └─ lwc-apex-call-implementation-guide.md
 - └─ lwc-error-handling-guide.md
 - └─ lwc-request-loading-guide.md
 - └─ pather_lwc_state_management_checklist-guide.md
 - └─ lwc-css-design-guide.md
 - └─ soql-exclude-deleted-guide.md

5. Task-skills and their guide dependencies

For each task-skill below: its name, what it does, and every guide it depends on — the guide's type and the simple condition that governs it. A required guide carries a trigger (when it is read); an optional guide carries the yes/no question that gates it.

create-new-parent-lwc-component

Pather (parent) router. Locks the pather's name + purpose once, then loops offering ADD NEW FUNCTIONALITY and HANDLE CHILD EVENT, delegating each to the matching sub-skill below. It emits no code itself, so it declares no guides directly — every guide lives in the sub-skill it routes to.

Direct guide dependencies: none.

add-interactive-with-data-persistence-functionality-in-pather

Six-step interview (sub-components, user stories, behavior, validations, data state, reusable base component) plus Apex-method resolution and wire-style decision, for adding an interactive data-persisting functionality to an existing parent LWC.

Guide	Type	When read / question
lwc-path-architecture-guide	required	Always — settle LWC layering before any interview question.

apex-method-resolution-guide	required	Always — resolve which Apex method backs the functionality (3 branches).
lwc-apex-call-implementation-guide	required	Always — decide @wire vs imperative call style from cacheability.
lwc-error-handling-guide	required	Always — route every failure through ShowToastEvent, not an inline banner.
pather_lwc_state_management_checklist-guide	required	Always — walk the state checklist (Rules 0-7) before emitting code.
apex-path-architecture-guide	required	When a new Apex method is implemented (skip if none is created).
soql-exclude-deleted-guide	optional	Q: "Apply the soft-delete filter?" — only when a new SELECT on a RecordStatus__c object is created.
lwc-css-design-guide	optional	Q (Step 5): "Apply the project's CSS design system now?"

lwc-parent-event-handler-generator

Per-event interview for wiring a parent LWC to handle CustomEvents dispatched by a child (state identification -> Apex resolution -> concurrent writes -> visibility urgency -> expand timing), producing state-coherent handlers and the correct @wire/imperative call style.

Guide	Type	When read / question
-------	------	----------------------

lwc-path-architecture-guide	required	Always — settle LWC layering before any interview question.
apex-method-resolution-guide	required	Always — resolve which Apex method backs each event handler.
lwc-apex-call-implementation-guide	required	Always — decide @wire vs imperative + refreshApex.
lwc-error-handling-guide	required	Always — route every failure through ShowToastEvent.
lwc-request-loading-guide	required	Always — wire the handler's Apex call to the isLoading spinner overlay.
pather_lwc_state_management_checklist-guide	required	Always — walk the state checklist (Rules 0-7) before emitting code.
apex-path-architecture-guide	required	When a new Apex method is implemented (skip if none is created).
soql-exclude-deleted-guide	optional	Q: "Apply the soft-delete filter?" — only when a new SELECT on a RecordStatus__c object is created.
lwc-css-design-guide	optional	Q (Step 5): "Apply the project's CSS design system now?"

create-new-child-lwc-component

Per-sub-component interview (user stories, behavior, validations, @api data state, reusable base component) for scaffolding a brand-new child LWC. Children are

presentation-only: no Apex, no @api mutation, draft state with _ + @track, lowercase events dispatched upward.

Guide	Type	When read / question
lwc-path-architecture-guide	required	Always — settle LWC layering before any interview question.
lwc-error-handling-guide	required	Always — route every child failure through ShowToastEvent.
lwc-css-design-guide	optional	Q (Step 5): "Apply the project's CSS design system now?"

add-functionality-in-child-lwc-component

Same per-sub-component interview, but extending an existing child LWC in place with a new sub-component or new functionality, preserving the existing sub-components, getters, handlers, and dispatched events.

Guide	Type	When read / question
lwc-path-architecture-guide	required	Always — settle LWC layering before any interview question.
pather_lwc_state_management_checklist-guide	required	Always — the parent handling these events must pass the state checklist.
lwc-error-handling-guide	required	Always — route every failure through ShowToastEvent.
lwc-css-design-guide	optional	Q (Step 5): "Apply the project's CSS design system now?"

Shared sub-step

shared/ask-user-for-lwc-validation.md is a leaf shared sub-step (not a guide) for the "Validation rules" interview question. All four functionality/child task-skills load it verbatim instead of inlining the validation branches, so the rule lives in one file.

Project-wide cross-cutting skills (triggered from CLAUDE.md)

These are not listed in any task-skill's Mandatory-guides step. CLAUDE.md decides when they fire, so they stay in one place and out of the per-task dependency graph.

Skill	When it fires
guard/apex-governor-limit-guard	Detect — when a bulk query/command (SELECT or DML over >1 record) is written or edited.
performance/apex-bulk-soql	Rewrite — same trigger, or when the guard flags an N+1 pattern.
performance/apex-method-monitor	Measure — after creating/editing an @AuraEnabled method that does SOQL/SOSL/DML (asks first).
guard/interview-discipline	On every iteration that has a question step.
guide/soql-exclude-deleted-guide	For any SELECT on a RecordStatus__c object (mandatory via CLAUDE.md; also surfaced as the optional question inside the parent functionality skills).

6. How the leaf-guide design resolves the spaghetti dependency

The earlier design let task-skills inline their sub-steps and reference one another, so a single rule lived in many copies and a new task could force edits across the others. The current design removes that with a few invariants:

Three layers, edges point down. orchestrator -> task-skill -> guide. There is no fourth hop and no sideways edge between task-skills.

Guides are leaves. A guide never imports another guide or a task-skill, so the graph has no cycles and no mesh.

One rule, one home. A shared rule lives in exactly one guide; N task-skills reference it by listing it in their Mandatory-guides step, not by copying it — change it once and every caller gets the change.

Adding a task-skill is additive. A new task-skill only adds edges to existing leaf guides; it never forces an edit in a sibling task-skill.

Cross-cutting rules are lifted out. performance/ and guard/ are triggered from CLAUDE.md rather than wired into the task graph, so they do not multiply edges across task-skills.